

โปรเจกต์จบการศึกษา

Production Control Tower

ระบบติดตามประสิทธิภาพการผลิต อุปกรณ์ และการบำรุงรักษา
สำหรับโรงงานอุตสาหกรรมการผลิตแบบมีสายการผลิต

Next.js 15 + React 19

TypeScript ทั้งหมด

Supabase + RLS

สิทธิ์บังคับที่ฐานข้อมูล

37 unit tests

+ 13 RLS assertions

ชื่อ - นามสกุล · รหัสนักศึกษา · อาจารย์ที่ปรึกษา

ทำไมต้องมีระบบนี้

สภาพเดิม — ข้อมูลกระจายเกินกว่าจะตัดสินใจได้

- ข้อมูล OEE อยู่ในไฟล์ Excel ของหัวหน้ากะ ต่างแผนกไม่เห็นชุดเดียวกัน
- เครื่องจักรหยุดแล้วไม่มีใครรู้ว่าเป็นเพราะอะไร หรือจะเสร็จเมื่อไหร่
- ประวัติการซ่อมอยู่ในกระดาษ ค้นย้อนหลังไม่ได้
- อะไหล่ขาดตอนงานด่วน เพราะไม่รู้ว่าจะอะไรกำลังจะหมด
- ไม่มีใครรู้ว่าใครทำอะไรไปแล้ว ทำให้แก้ข้อมูลชนกัน

สิ่งที่ต้องการ

- **เห็นภาพรวม OEE ของทุกสายในหน้าเดียว**
- บันทึกเหตุหยุดเครื่องพร้อมเหตุผลและผู้รับผิดชอบ
- วางแผนงานซ่อมได้ก่อนเครื่องพังจริง (งานตามรอบ)
- รู้สถานะอะไหล่ก่อนสั่งซื้อ
- ตรวจสอบย้อนหลังได้ว่าใครเปลี่ยนอะไร
- บทบาทชัดเจน คนละสิทธิ์กันจริง ไม่ใช่แค่ซ่อนปุ่ม

หัวใจของงาน: ทุกอย่างต้องคำนวณจากข้อมูลจริงในฐานข้อมูล ไม่ใช่ค่าที่พิมพ์กรอกเอง

ระบบนี้ทำอะไรได้บ้าง

แดชบอร์ด OEE

สรุป A/P/Q แยกรายสาย พร้อมกราฟ
ย้อนหลัง

ทรัพย์สิน

ทะเบียนเครื่องจักร สถานะ ประวัติการ
ซ่อม

ใบงานบำรุง

งานถูกต้อง งานตามรอบ งานสอบเทียบ

หยุดเครื่อง

บันทึกเหตุ แจ้งเตือน คำนวณ MTTR

คลังอะไหล่

จอง เบิก กันของติดลบ

รายงาน

ส่งออก CSV ได้ทุกหน้า

บันทึกการกระทำ

ประวัติทุกครั้งที่มีการแก้ไข

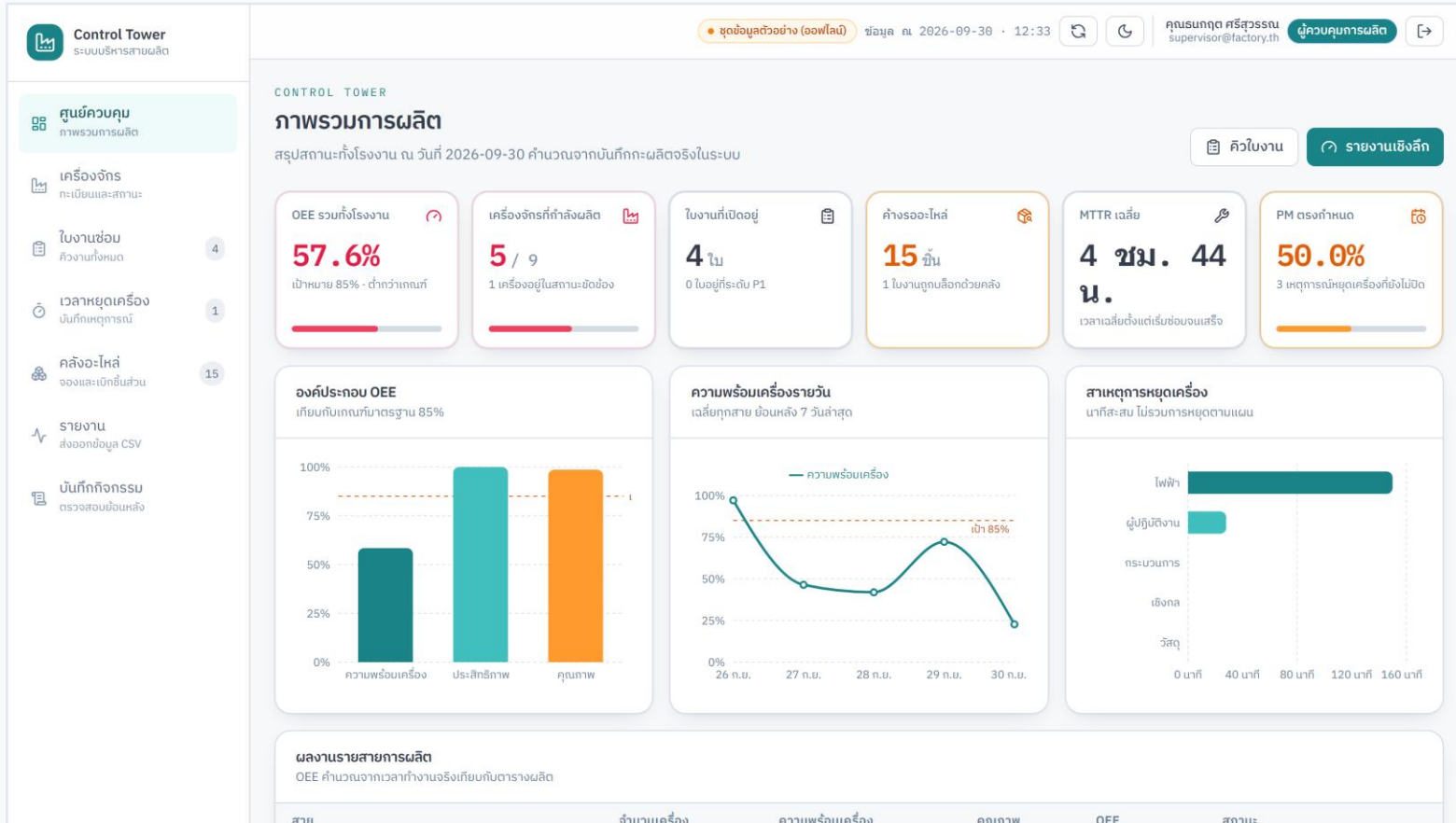
ข้อมูลตัวอย่าง

2 โรงงาน · 3 สาย
10 เครื่อง · 8 ใบงาน
8 อะไหล่ · 30 รอบการผลิต

สิ่งที่ระบบนี้ไม่ทำ — บอกไว้ตรง ๆ ไม่งอน

- ไม่ได้อ่านสัญญาณจาก PLC จริง เวลาหยุดเครื่องยังให้คนกรอกเอง
- ยังไม่ต่อกับ ERP หรือระบบสั่งซื้ออะไหล่ของโรงงาน
- ไม่ได้ทำหน้าจัดการผู้ใช้ (เพิ่มผู้ใช้ต้องเขียน SQL เอง)

แดชบอร์ด — เห็นภาพรวมทั้งโรงงานในหน้าเดียว



OEE รวมทุกสาย

57.6%

ของดี 47,259 ชิ้น · ของเสีย 656 ชิ้น

Availability 58.4% · Quality 98.6%

จุดที่ระบบชี้ให้เห็น

- 3 เหตุหยุดเครื่องที่ยังไม่ปิด รวม 253 นาที
- 3 อะไหล่เหลือต่ำกว่าจุดสั่งซื้อ
- MTTR 284 นาที · งานตามรอบทำได้ 50%

ภาพหน้าจอจริงจากระบบ ผู้ใช้บทบาทผู้ดูแล

OEE คำนวณอย่างไร และได้เท่าไรจริง

Availability

เวลาที่เครื่องเดินจริง ÷ เวลาที่วางแผนไว้

58.4%

Performance

ผลผลิตตามทฤษฎี ÷ ผลผลิตจริง

100.0%

Quality

ของดี ÷ ของทั้งหมด

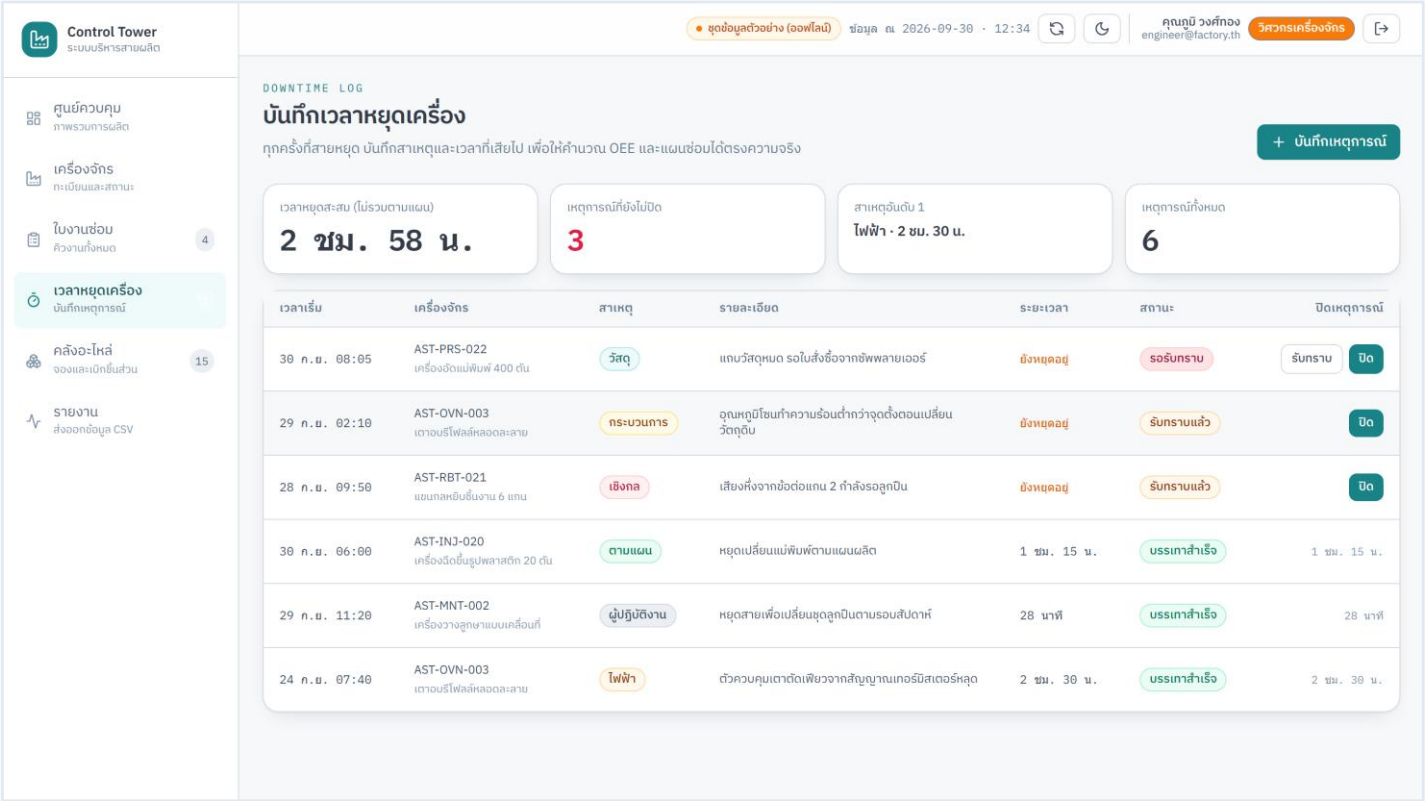
98.6%

OEE = Availability × Performance × Quality = **57.6%** ทั้งระบบ

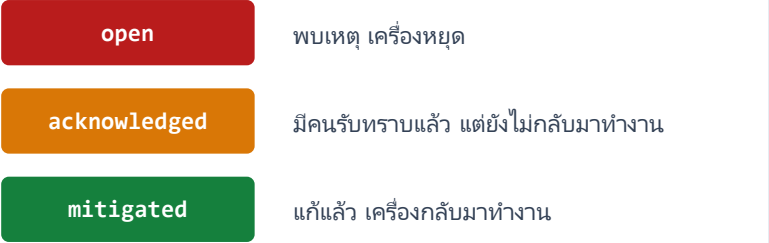
สายการผลิต	Availability	Quality	OEE	เวลาที่หยุด
LN-A สายวางลูกขา SMT	57.3%	99%	56.7%	42.7%
LN-B สายเติมวงจรและทดสอบ	74.1%	98.7%	73.1%	25.9%
LN-C สายฉีดขึ้นรูปพลาสติก	56%	98.4%	55.1%	44%
รวม	58.4%	98.6%	57.6%	41.6%

ตัวเลขทั้งหมดมาจากการรันค่าสแกนจริงบนชุดข้อมูลตัวอย่าง ไม่ได้เขียนขึ้นมือ

จัดการเหตุหยุดเครื่อง



สถานะของเหตุหยุดเครื่อง



ตัวเลขจากข้อมูลจริง

1 เหตุ ยังไม่มีใครรับทราบ 2 เหตุ รับทราบแล้ว
3 เหตุ ปิดเรียบร้อยแล้ว · รวม 253 นาที

MTTR เฉลี่ย 284 นาที

ระบบ ไม่ปิดเหตุให้อัตโนมัติ — ต้องมีคนยืนยันว่าเครื่องกลับมาทำงานจริง

อะไหล่ — ก้นงานค้างเพราะของขาด

Control Tower

ระบบบริหารสายผลิต

ศูนย์ควบคุม

ภาพรวมการผลิต

เครื่องจักร

ทะเบียนและสถานะ

ใบงานซ่อม

คิวงานทั้งหมด

4

เวลาหยุดเครื่อง

บันทึกเหตุการณ์

1

คลังอะไหล่

จองและเบิกชิ้นส่วน

รายงาน

ส่งออกข้อมูล CSV

ดูข้อมูลตัวอย่าง (ออฟไลน์)

ข้อมูล ณ 2026-09-30 · 12:34

คุณภูมิ วงศ์ทอง

engineer@factory.th

วิศวกรเครื่องจักร

➔

SPARES STORE

คลังอะไหล่สำรอง

ติดตามยอดคงเหลือ การจอง และการเบิกใช้เข้ากับใบงานซ่อมบำรุง

+ เพื่ออะไหล่

มูลค่าสต็อกรวม

฿102,320

รายการที่ต้องสั่งซื้อ

3

รายการในคลัง

8

ค้นหาด้วย SKU หรือชื่ออะไหล่

เฉพาะที่ต้องสั่งซื้อ 8/8 รายการ

อะไหล่	คงเหลือ	จองไว้	ว่างใช้	จุดสั่งซื้อ	ความต้องการจากงาน	ราคา/ชิ้น	ปรับยอด
สายพานตาข่าย 8 มม. กว้าง 1200 BLT-CONV-08 · ชิ้น D-01-01 · นำเข้า 21 วัน	14	0	14	6	—	฿2,100	- 14 +
สกรูหกหัว M12x60 เกรด 12.9 BLT-M12-60 · ชิ้น A-02-01 · นำเข้า 3 วัน	240	20	220	60	8 ชิ้น	฿23	- 240 +
ลูกปืนแบบหมุน 6205-2RS ปิดฝาสองด้าน BRG-6205-2RS · ชิ้น A-01-03 · นำเข้า 5 วัน	48	4	44	10	4 ชิ้น	฿185	- 48 +
เทอร์มิสเตอร์ NTC 100K 0395 NTC-100K-B395 · ชิ้น B-03-05 · นำเข้า 14 วัน	6	0	6	ต้องสั่งซื้อ	—	฿320	- 6 +
น้ำมันเกียร์ ISO VG 68 (18 ลิตร) OIL-ISO-68 · ชิ้น A-05-04 · นำเข้า 7 วัน	22	2	20	8	1 ชิ้น	฿1,750	- 22 +
เซลล์แรงดัน 500 กรัม SNS-FT-500 · ชิ้น C-01-07 · นำเข้า 35 วัน	3	0	3	ต้องสั่งซื้อ	—	฿2,890	- 3 +
พัดลมเทอร์โบ 230V 23W							

กฎที่ระบบบังคับ

- ของที่จองไว้แล้วห้ามเบิกเกิน
- ห้ามเบิกจนเหลือน้อยกว่าจุดสั่งซื้อ
- เบิกไม่ได้ถ้าของหมด — ต้องสั่งซื้อก่อน
- ทุกครั้งที่เบิก ต้องผูกกับใบงาน

สถานะคลัง

ต่ำกว่าจุดสั่งซื้อ

3 / 8 รายการ

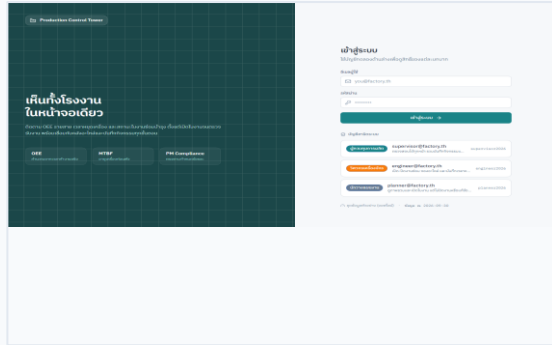
มูลค่าของคงคลัง

102,320 บาท

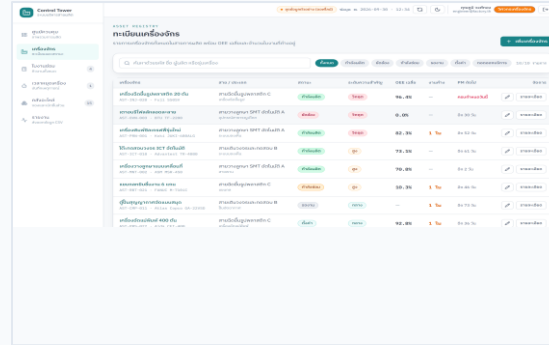
ใบงาน 4 ใบรอซ่อม · งานตามรอบ 3 ใบ · งานสอบเทียบ 1 ใบ

หน้าจอทั้งหมด

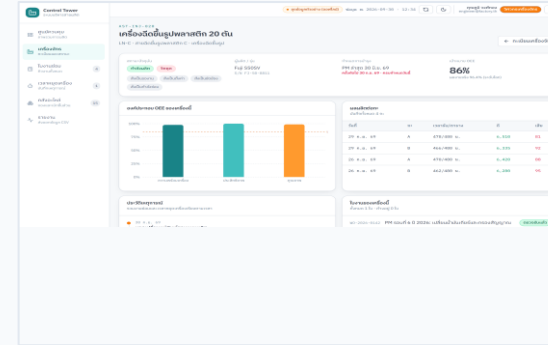
ระบบมี 12 หน้า ผ่านการทดสอบทุกหน้า



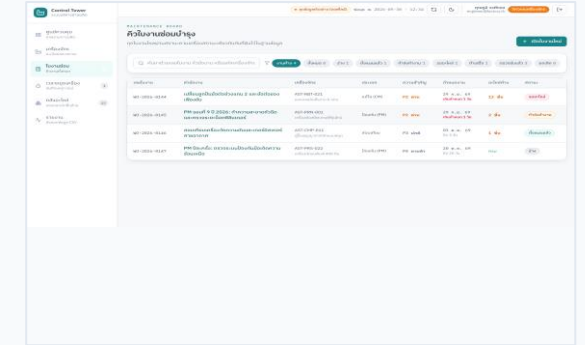
หน้าเข้าสู่ระบบ



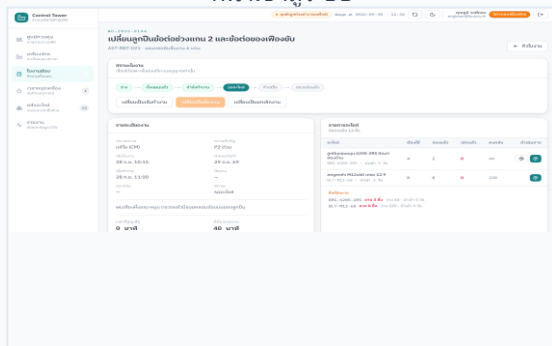
ทะเบียนเครื่องจักร



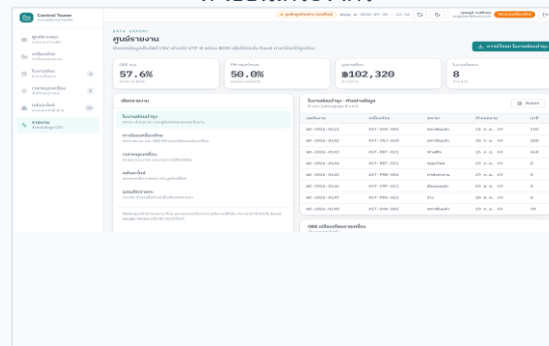
รายละเอียดเครื่อง



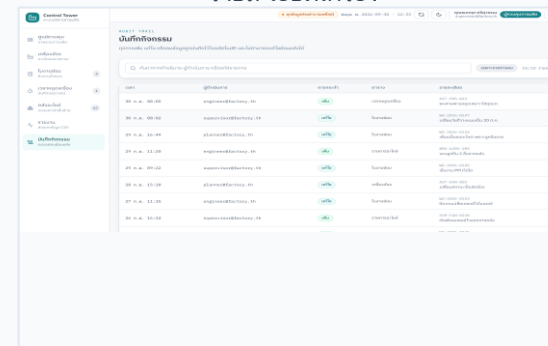
โรงงานบำรุง



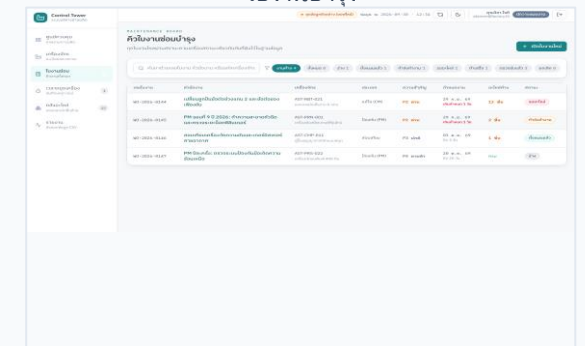
รายละเอียดใบงาน



รายงานและส่งออก CSV



ประวัติการแก้ไข



มุมมองผู้วางแผน

มีอีก: แดชบอร์ดผู้ดูแล · แดชบอร์ดวิศวกร · แดชบอร์ดผู้วางแผน · โหมดมืด · มุมมองมือถือ

ข้อมูลเดินทางอย่างไร

ชั้นแสดงผล

React + Next.js · 12 หน้า · Tailwind + Recharts



ชั้นตรรกะ

สูตร OEE · กติกาเปลี่ยนสถานะ · ตารางสิทธิ์ตามบทบาท



ชั้นข้อมูล

Supabase (PostgreSQL) · 11 ตาราง · 25 สิทธิ์ระดับแถว · 15 trigger

ทำงานได้แม้ไม่มีเซิร์ฟเวอร์

- ถ้ายังไม่ได้ต่อฐานข้อมูล ระบบจะใช้ชุดข้อมูลตัวอย่างในเครื่อง
- เหมาะกับการสาธิตและทดสอบหน้าจอโดยไม่ต้องตั้งค่าอะไร

การเขียนข้อมูล

- หน้าจออัปเดตพื้นที่ ไม่ต้องรอเครือข่าย
- ถ้าฐานข้อมูลปฏิเสธ ระบบย้อนกลับหน้าจอให้ตรงเดิม
- บันทึกการสุ่มอุปกรณ์เป็นรหัสสุ่ม ไม่ใช่ UUID

สิทธิ์บังคับที่ฐานข้อมูล ไม่ใช่แค่ซ่อนปุ่ม

ตัวอย่างที่ทดสอบแล้ว — ผู้ใช้ที่พิสูจน์ตัวตนแล้วแต่ส่งคำขอที่ไม่มีสิทธิ์ ฐานข้อมูลปฏิเสธเอง

ผู้ดูแล

admin

- ✓ ดูและแก้ไขข้อมูลทุกอย่าง
- ✓ จัดการผู้ใช้
- ✓ อ่านประวัติการแก้ไขทั้งหมด
- ✗ แก้หรือลบประวัติ — ไม่ได้

วิศวกร

engineer

- ✓ แก้สถานะเครื่องจักร
- ✓ เปิดและปิดโบราณซ่อม
- ✓ บันทึกเหตุหยุดเครื่อง
- ✗ เพิ่มหรือแก้ไขข้อมูลอะไหล่ — ไม่ได้

ผู้วางแผน

planner

- ✓ ดูข้อมูลได้ทั้งหมด
- ✓ วางแผนงานตามรอบ
- ✗ เปิดงานซ่อมฉุกเฉิน — ไม่ได้
- ✗ แก้เครื่องจักร — ไม่ได้

เจาะลึก: ผู้ใช้ยังไม่ล็อกอินจะได้สิทธิ์เท่าผู้วางแผน ไม่ใช่สิทธิ์สูงสุด — ถ้าตั้งค่าผิดจะเป็นการปิดทางเข้า ไม่ใช่เปิดกว้าง

บั๊กที่ 1 — โค้ดที่อ่านผ่าน แต่รันไม่ผ่าน

ทั้งโปรเจกต์เขียนเสร็จ ทุกเทสต์ผ่าน แต่พอเอา SQL ไปรันกับฐานข้อมูลจริงกลับพังตั้งแต่บรรทัดแรก

ก่อนแก้

```
-- ฟังก์ชันอยู่ตรงนี้
create function authz.current_role()
returns app_role as $$
    select role from public.app_users
    where user_id = auth.uid();
$$ language sql;

-- แต่ตาราง app_users ถูกสร้างทีหลัง
```

หลังแก้

```
-- สร้างตารางก่อน
create table public.app_users (...);

-- แล้วค่อยสร้างฟังก์ชัน
create function authz.current_role()
returns app_role as $$
    select role from public.app_users
    where user_id = auth.uid();
$$ language sql;
```

บอกได้ตรง ๆ ว่าเป็นบั๊กอะไร

PostgreSQL ตรวจการอ้างถึงตารางข้างในฟังก์ชันตอนสร้าง ไม่ใช่ตอนเรียกใช้ การอ่านโค้ดด้วยตาจะเห็นว่า “เฮ้ย อ้างถึงตารางที่ยังไม่มี” แต่ถ้าไม่เคยรันจริงก็จะมีวันรู้ว่ามันพัง

วิธีแก้ที่ถาวร: ให้เขียน schema ให้รันซ้ำได้ และมีชุดทดสอบที่รัน SQL จริงทุกครั้ง

บั๊กที่ 2 และ 3 — จุดที่อ่านโค้ดไม่ออก

บั๊กที่ 2 — ตัวพิมพ์ใหญ่เล็กไม่ตรงกัน

trigger ได้ค่า **INSERT** (ตัวพิมพ์ใหญ่)

แต่ชนิดข้อมูล enum เก็บเป็น

insert (ตัวพิมพ์เล็ก)

```
ก่อน: tg_op::trail_action    แก้: lower(tg_op)::trail_action
```

บั๊กที่ 3 — ข้อมูลกับกฎทางธุรกิจขัดกันเอง

กฎเดิม: ถ้ายังไม่ปิดเหตุ ต้องมีเวลาสิ้นสุด

แต่ข้อมูลจริงมีเหตุที่

“รับทราบแล้ว เครื่องยังไม่กลับมา” ซึ่งยังไม่จบจริง

แก้โดยบังคับเวลาสิ้นสุดเฉพาะสถานะที่เครื่องกลับมาทำงานแล้วเท่านั้น

บั๊กที่ 3 สำคัญที่สุด เพราะมันไม่ใช่บั๊กทางเทคนิค

ตัวไหนผิด — SQL หรือข้อมูล — ขึ้นอยู่กับความหมายทางธุรกิจ ไม่ใช่ไวยากรณ์ ถ้าผมเดาเองแล้วแก้ผิดฝั่ง ระบบจะทำงานผิดแบบเงียบ ๆ ไม่มี error ให้เห็น

ผลที่ได้: ตอนนี้มี การทดสอบอัตโนมัติที่ยกคอนเทนเนอร์ฐานข้อมูลจริงขึ้นมารันทุกครั้ง บั๊กแบบนี้จะไม่หลุดออกไปได้อีก

ทดสอบอะไรไปบ้าง และผ่านหมดหรือยัง

✓	ตรวจชนิดข้อมูล	ผ่าน	ไม่มี error	✓	ตรวจกฎการเขียนโค้ด	ผ่าน	ไม่มี warning
✓	ทดสอบสูตรและกติกา	37 / 37	หน่วยทดสอบ	✓	สร้างไฟล์จริง	ผ่าน	12 หน้า
✓	รัน SQL กับฐานข้อมูลจริง	ผ่าน	PostgreSQL 16	✓	ทดสอบสิทธิ์ระดับแถว	13 / 13	RLS
—	รันกับ Supabase จริง	ยังไม่ได้	รอการเชื่อมต่อ	—	ทดสอบผู้ใช้หลายคนพร้อมกัน	ยังไม่ได้	ต้องมีคนจริง 2 คน

คำสั่งเดียวที่ยืนยันได้ทั้งหมด

npm run db:test

ยกฐานข้อมูลจริงขึ้นมาใหม่ทุกครั้ง ใส่ข้อมูลตัวอย่าง แล้วทดสอบสิทธิ์ที่ละบทบาท ใช้เวลาไม่ถึงหนึ่งนาที ปิดท้ายด้วยการลบคอนเทนเนอร์ทิ้ง ไม่มีอะไรค้างในเครื่อง

สิ่งที่ยังทำไม่ได้ และจะทำต่ออย่างไร

ยังไม่ได้ต่อกับ Supabase จริง

ทดสอบกับฐานข้อมูลจริงแล้ว แต่ยังใช้ตัวจำลองแทนส่วนที่ Supabase เพิ่มเข้ามา

→ ต่อโปรเจกต์จริงแล้วรันซ้ำ

ยังไม่ได้ทดสอบหลายคนพร้อมกัน

ระบบเขียนแบบเร็วที่สุดแล้วย้อนกลับเมื่อพัง ถ้าสองคนแก้ข้อมูลเดียวกัน คนหลังจะทับคนแรก

→ ต้องทดสอบกับผู้ใช้จริง

เวลายังกรอกเอง ไม่ได้มาจาก PLC

ยังไม่ได้ต่อสัญญาณจริงผ่าน Modbus หรือ OPC UA

→ ต่อกับ PLC เมื่อมีอุปกรณ์จริง

เขียนข้อมูลเป็นรายการสั่ง ไม่มีธุรกรรม

ถ้าฐานข้อมูลปฏิเสธบางแถว ของที่เขียนไปแล้วจะไม่ถูกย้อนกลับ

→ ใช้ฟังก์ชันที่ห่อธุรกรรมเดียว

รหัสผ่านตัวอย่างเก็บเป็นข้อความธรรมดา

เหมาะกับการสาธิตเท่านั้น

→ ต้องลบตารางนี้ก่อนใช้จริง

ยังมีช่องโหว่ในไลบรารีที่ Next.js ดึงมา

อยู่ในขั้นตอน build ไม่กระทบผู้ใช้ปลายทาง แต่ควรอัปเดตเมื่อมีเวลา

→ อัปเดต Next.js เป็นเวอร์ชันใหม่

สรุป

ระบบทำงานได้จริง และตอนนี้ผมรู้แน่ชัดว่าอะไรยังไม่น่าเชื่อถือ

12 หน้า

OEE เครื่องจักร ใงาน หยุดเครื่อง อะไร่ล รายงาน และประวัติการแก้ไข

50 เคสทดสอบ

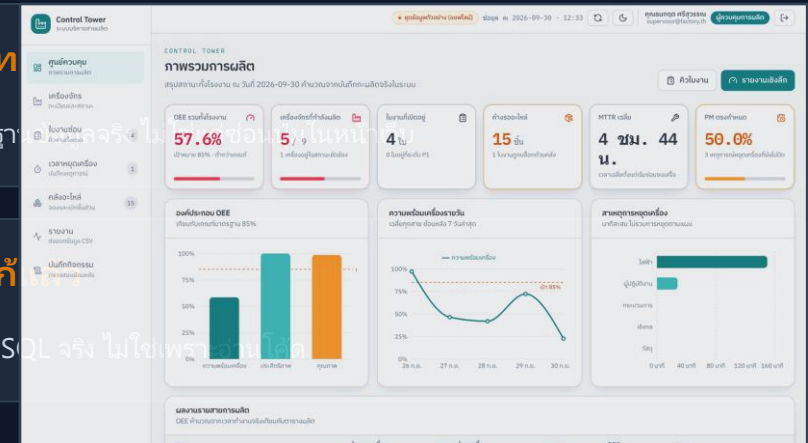
ทดสอบหน่วย 37 เคส และทดสอบสิทธิ์ 13 เคสกับฐานข้อมูลจริง

3 บทบาท

สิทธิ์บังคับที่ฐานข้อมูลจริง ไม่ใช้เพราะฐานข้อมูล

3 บั๊กที่แก้

เจอเพราะรัน SQL จริง ไม่ใช่เพราะรันโมด



ขั้นตอนที่สำคัญที่สุด: เชื่อมฐานข้อมูลจริง แล้วรันชุดทดสอบเดิมซ้ำอีกครั้ง

โหมดมืด

ขอบคุณที่รับฟังครับ

ถามได้ทุกข้อ